Politechnika Wrocławska

Wydział Elektorniki, Fotoniki i Mikrosystemów

# STEROWANIE PROCESAMI DYSKRETNYMI
# LABORATORIUM 1

Michał Sadecki 278317
Wojciech Zielinski 278327

Prowadząca:
dr inż. Agnieszka Wielgus

Wrocław, 17 marca 2026

# 1 Opis problemu

Rozważanym problemem jest szeregowanie zadań, a dokładniej problem $1|r_j|L_{max}$, czyli dla jednej maszyny pojawia się w danym czasie zadanie $j$, dla którego należy zminimalizować maksymalną nieterminowość. Należy opracować algorytmy, które rozwiążą ten problem, mając dane następujące parametry:

- $p_j$ - czas przetwarzania zadania przez maszynę
- $d_j$ - termin pożądany zakończenia zadania
- $C_j$ - rzeczywisty czas zakończenia zadania
- $L_j$ - nieterminowość, czyli z definicji $C_j - d_j$. Dla ujemnej wartości zadanie zostało ukończone przed czasem.

# 2 Zaimplementowane algorytmy

Implementację przeprowadzono w języku C++, ponieważ daje on dużą kontrolę nad implementacją, a z drugiej strony wysoki poziom abstrakcji.

Stworzono cztery klasy:

- `Task`, która reprezentuje zadanie
- `Permutation`, która tworzy permutacje
- `Solution`, która rozwiązuje problem dla danej permutacji
- `Problem`, której metody to rozważane algorytmy rozwiązań problemu

# 3 Opis algorytmów

Implementowane algorytmy różnią się stopniem skomplikowania, złożonością, czasem działania, trudnością w implementacji. Podczas pisania przeprowadzano szereg testów na przykładzie, który rozwiązano ręcznie.

## 3.1 Algorytmy w oparciu o sortowanie

W tych algorytmach sortuje się listę zadań po danym współczynniku, np. $r_j$ lub $d_j$. Dla posortowanych zadań rozwiązuje się rozważany problem. Napisana funkcja sortująca przyjmuje argument domyślny, w którym wskazywany jest parametr, po którym następuje sortowanie. Domyślnie sortowanie odbywa się po $d_j$. Pseudokod zaimplementowanego algorytmu znajduje się na listingu poniżej.

**Algorithm 1** Sortowanie po $d_j/r_j$

```
procedure SortAlgorithm(tasks, by_rj)
    if by_rj then
        sort perm by tasks[i].rj ascending
    else
        sort perm by tasks[i].dj ascending
    end if
    result ← solve(perm)
    return result
end procedure
```

## 3.2 Przegląd zupełny

Ten algorytm jest klasycznym przykładem metody zachłannej, czyli rozważane są wszystkie możliwości parametrów i rozwiązując problem dla każdego przypadku wybierany jest ten, który ma najmniejszą maksymalną nieterminowość.

**Algorithm 2** Przegląd zupełny

```
procedure Bruteforce(tasks, perm)
    best_sol ← ∞
    best_perm ← ∅
    repeat
        temp ← solve(perm)
        if temp < best_sol then
            best_sol ← temp
            best_perm ← perm
        end if
    until next_permutation(perm) is false
    perm ← best_perm return best_sol
end procedure
```

## 3.3 Własna implementacja

Opracowany algorytm opiera się na idei twierdzenia o zamianie zadań $(j,\ j+1)$. W tym przypadku zamieniane są wszystkie sąsiadujące zadania, np. 1–2, 3–4, 5–6, ..., (n−1)–n. Gdy rozwiązanie dla danej zamiany $(j, j+1)$ jest gorsze niż przed zamianą, to zamiana nie następuje. Algorytm jest szybki, a złożoność zależy od ilości zadań (`O(n)`). Wadą tego algorytmu jest mocna zależność od początkowego posortowania zadań. Pseudokod tego rozwiązania znajduje się na poniższym listingu.

**Algorithm 3** Własny algorytm (lokalna zamiana)

```
procedure OwnAlgorithm(tasks, n)
    for i ← 0 to n − 2 do
        L ← solve(perm)
        swap(perm[i], perm[i + 1])
        if solve(perm) > L then
            swap(perm[i], perm[i + 1])
        end if
    end for
    return solve(perm)
end procedure
```

## 3.4 Algorytm Schrage

Algorytm polega na sprawdzaniu wszystkich dostępnych zadań w danej chwili $t$ pod kątem parametru $d_j$. Jeżeli w chwili $t_1$ dostępne są zadania $k$, $l$, $m$, to wybierane jest to, które ma najmniejszy parametr $d_j$. Dla tego zadania wyznaczane jest $L$, a następnie operacja powtarza się dla kolejnych dostępnych zadań. Można na to spojrzeć, jak na kolejkę priorytetową z priorytetem $d_j$, do której dodawane są elementy z $r_j \leq t_j$. Pseudokod zaimplementowanego algorytmu znajduje się na listingu poniżej.

**Algorithm 4** Algorytm Schrage

```
procedure Schrage(tasks)
    all_tasks ← PriorityQueue sorted by r_j ascending
    available ← PriorityQueue sorted by d_j ascending
    L_max ← −∞,   time ← 0,   last_flag ← false
    temp_task ← all_tasks.pop()
    time ← max(time, tasks[temp_task].r_j)
    while all_tasks is not empty or available is not empty do
        while tasks[temp_task].r_j ≤ time and not last_flag do
            available.push(temp_task)
            if all_tasks is not empty then temp_task ← all_tasks.pop()
            else last_flag ← true
            end if
        end while
        current_task ← available.pop()
        time ← time + tasks[current_task].p_j
        L_max ← max(L_max, time − tasks[current_task].d_j)
        if tasks[temp_task].r_j > time and available is empty then
            time ← max(time, tasks[temp_task].r_j)
        end if
    end while
    return L_max
end procedure
```

### 3.5 Algorytm Schrage z wywłaszczaniem

Algorytm jest analogiczny, jak powyżej, ale posiada własność wywłaszczania. Gdy pojawia się zadanie w chwili $t_j$, to sprawdzane jest, czy to zadanie ma $d_j$ mniejszy niż aktualne rozważane zadanie. Gdy tak jest, to wykonanie aktualnego zadania jest przerywane (wraca do kolejki), a w jego miejsce wskakuje zadanie o najmniejszym $d_j$. Dalej procedura jest analogiczna. Pseudokod tego rozwiązania znajduje się na listingu.

**Algorithm 5** Algorytm wywłaszczający Schrage

```
procedure SchragePreemptive(tasks)
    ...
    while all_tasks is not empty or available is not empty do
        while tasks[temp_task].r_j ≤ time and not last_flag do
            available.push(temp_task)
            ...
        end while
        current ← available.top()
        if tasks[temp_task].r_j < (time + tasks[current].p_j) then
            and not last_flag then
            tasks[current].p_j ← tasks[current].p_j − (tasks[temp_task].r_j −
time)
            time ← tasks[temp_task].r_j
        else
            time ← time + tasks[current].p_j
            L_max ← max(L_max, time − tasks[current].d_j)
            available.pop()
        end if
        if tasks[temp_task].r_j > time and available is empty then
            time ← max(time, tasks[temp_task].r_j)
        end if
    end while
     return L_max
end procedure
```

# 4 Testy

Testy zostały przeprowadzone na zbiorach danych załączonych w instrukcji. Są to instancje zebrane przez Uniwersytet w Gandawie (Belgia). Rezultaty przedstawiono w poniższej tabeli.

| Rozmiar instancji | Wartość kryterium sortowania po r_j | Czas działania algorytmu sortowania po r_j [µs] | Wartość kryterium sortowania po d_j | Czas działania algorytmu sortowania po d_j [µs] | Wartość kryterium przeglądu zupełnego | Czas działania przeglądu zupełnego [µs] | Wartość kryterium algorytmu własnego | Czas działania algorytmu własnego [µs] | Wartość kryterium algorytmu Schrage | Czas działania algorytmu Schrage [µs] | Wartość kryterium algorytmu Schrage wywłaszczającego | Czas działania algorytmu Schrage z wywłaszczaniem [µs] | Nazwa pliku z danymi testowymi |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 10 | 62 | 74 | 46 | 34 | 45 | 2632881 | 62 | 187 | 45 | 150 | 45 | 118 | test.txt |
| 9 | 41 | 89 | 32 | 21 | 32 | 264479 | 49 | 71 | 32 | 109 | 32 | 77 | test2.txt |
| 100 | 4389 | 155 | 1686 | 125 | - | - | 5340 | 4294 | 1019 | 410 | 992 | 409 | 100_Independent0291 |
| 100 | 3828 | 171 | 3759 | 140 | - | - | 4539 | 4436 | 2723 | 412 | 2708 | 225 | 100_Independent0333 |
| 100 | 2103 | 351 | -1497 | 335 | - | - | 2523 | 10984 | -1520 | 1070 | -1571 | 1221 | 100_Independent0420 |
| 100 | 3632 | 250 | 2324 | 217 | - | - | 5281 | 2014 | 1081 | 170 | 1036 | 166 | ../test/100_Independent0600 |
| 100 | 2875 | 261 | 1755 | 116 | - | - | 4007 | 1596 | 739 | 135 | 728 | 153 | ../test/100_Independent0601 |
| 100 | 3421 | 144 | 2219 | 74 | - | - | 5673 | 1333 | 781 | 113 | 700 | 157 | ../test/100_Independent0602 |
| 100 | 3502 | 78 | 2352 | 78 | - | - | 4894 | 1541 | 1146 | 114 | 1139 | 214 | ../test/100_Independent0603 |
| 100 | 3179 | 72 | 2114 | 55 | - | - | 4938 | 1253 | 964 | 101 | 952 | 197 | ../test/100_Independent0604 |
| 100 | 3226 | 56 | 2493 | 54 | - | - | 4584 | 1602 | 1138 | 102 | 1110 | 164 | ../test/100_Independent0605 |
| 100 | 3422 | 56 | 2354 | 45 | - | - | 5470 | 1218 | 1001 | 122 | 975 | 128 | ../test/100_Independent0606 |
| 100 | 3680 | 44 | 2452 | 45 | - | - | 4361 | 1143 | 1233 | 101 | 1157 | 128 | ../test/100_Independent0607 |
| 100 | 3443 | 44 | 2472 | 44 | - | - | 5352 | 1173 | 1100 | 119 | 1095 | 109 | ../test/100_Independent0608 |
| 100 | 2977 | 47 | 2126 | 42 | - | - | 5334 | 1148 | 788 | 102 | 734 | 109 | ../test/100_Independent0609 |
| 100 | 3035 | 35 | 1043 | 47 | - | - | 4759 | 1169 | 860 | 99 | 810 | 136 | ../test/100_Independent0610 |
| 100 | 3401 | 38 | 1336 | 60 | - | - | 5234 | 1181 | 1039 | 104 | 1015 | 143 | ../test/100_Independent0611 |
| 100 | 3527 | 37 | 1138 | 51 | - | - | 5431 | 1200 | 1002 | 102 | 940 | 175 | ../test/100_Independent0612 |
| 100 | 2936 | 47 | 1300 | 36 | - | - | 4424 | 1143 | 504 | 98 | 489 | 110 | ../test/100_Independent0613 |
| 100 | 3368 | 42 | 1644 | 33 | - | - | 4901 | 1430 | 1105 | 98 | 1058 | 111 | ../test/100_Independent0614 |
| 100 | 2896 | 43 | 1308 | 31 | - | - | 5232 | 1158 | 887 | 128 | 797 | 111 | ../test/100_Independent0615 |
| 100 | 3364 | 31 | 1285 | 32 | - | - | 5270 | 1150 | 973 | 99 | 915 | 111 | ../test/100_Independent0616 |
| 100 | 3385 | 46 | 965 | 32 | - | - | 5551 | 1159 | 762 | 97 | 714 | 109 | ../test/100_Independent0617 |
| 100 | 2405 | 31 | 750 | 32 | - | - | 4287 | 1917 | 499 | 122 | 444 | 175 | ../test/100_Independent0618 |
| 100 | 2394 | 32 | 512 | 32 | - | - | 5132 | 1946 | 469 | 98 | 390 | 112 | ../test/100_Independent0619 |
| 100 | 3099 | 53 | 853 | 31 | - | - | 4375 | 1984 | 787 | 98 | 785 | 112 | ../test/100_Independent0620 |
| 100 | 3121 | 31 | 1097 | 66 | - | - | 4120 | 1170 | 985 | 98 | 933 | 109 | ../test/100_Independent0621 |
| 100 | 3659 | 32 | 933 | 68 | - | - | 5105 | 1153 | 948 | 101 | 933 | 110 | ../test/100_Independent0622 |
| 100 | 3663 | 32 | 1051 | 44 | - | - | 5100 | 1543 | 1091 | 117 | 1051 | 109 | ../test/100_Independent0623 |
| 100 | 3182 | 30 | 680 | 42 | - | - | 4762 | 1150 | 641 | 124 | 639 | 110 | ../test/100_Independent0624 |
| 100 | 2630 | 31 | 294 | 30 | - | - | 4807 | 1246 | 305 | 99 | 294 | 116 | ../test/100_Independent0625 |
| 100 | 3304 | 33 | 1322 | 30 | - | - | 4687 | 1157 | 1078 | 95 | 1060 | 107 | ../test/100_Independent0626 |
| 100 | 3119 | 32 | 1080 | 33 | - | - | 5040 | 1147 | 980 | 96 | 963 | 110 | ../test/100_Independent0627 |
| 100 | 2875 | 44 | 825 | 31 | - | - | 4768 | 1697 | 705 | 102 | 669 | 111 | ../test/100_Independent0628 |
| 100 | 3060 | 43 | 507 | 43 | - | - | 5448 | 1159 | 526 | 99 | 507 | 110 | ../test/100_Independent0629 |
| 100 | 2308 | 30 | 2365 | 34 | - | - | 4236 | 1258 | 25 | 148 | 25 | 182 | ../test/100_Independent0650 |
| 100 | 1903 | 31 | 1926 | 33 | - | - | 3486 | 1178 | 45 | 97 | 45 | 111 | ../test/100_Independent0651 |
| 100 | 2190 | 32 | 2034 | 32 | - | - | 4608 | 1213 | 47 | 96 | 47 | 140 | ../test/100_Independent0652 |
| 100 | 1926 | 31 | 1960 | 32 | - | - | 4121 | 1179 | 48 | 95 | 48 | 112 | ../test/100_Independent0653 |
| 100 | 1822 | 32 | 1928 | 31 | - | - | 3498 | 1205 | 62 | 98 | 62 | 113 | ../test/100_Independent0654 |
| 100 | 2205 | 43 | 2345 | 36 | - | - | 3382 | 1301 | 33 | 118 | 33 | 111 | ../test/100_Independent0655 |
| 100 | 2354 | 47 | 2247 | 32 | - | - | 4010 | 1173 | 66 | 119 | 66 | 111 | ../test/100_Independent0656 |
| 100 | 2054 | 41 | 1815 | 32 | - | - | 4019 | 1174 | 74 | 117 | 64 | 142 | ../test/100_Independent0657 |
| 100 | 2094 | 32 | 1997 | 32 | - | - | 3281 | 1390 | 43 | 118 | 43 | 111 | ../test/100_Independent0658 |
| 100 | 2376 | 43 | 2424 | 33 | - | - | 3813 | 1293 | 84 | 119 | 84 | 170 | ../test/100_Independent0659 |
| 100 | 2548 | 32 | 1120 | 31 | - | - | 3833 | 1160 | -18 | 99 | -69 | 113 | ../test/100_Independent0660 |
| 100 | 2023 | 32 | 719 | 31 | - | - | 3756 | 1198 | -379 | 97 | -423 | 111 | ../test/100_Independent0661 |
| 100 | 1997 | 31 | 1355 | 45 | - | - | 3709 | 1335 | -112 | 97 | -154 | 111 | ../test/100_Independent0662 |
| 100 | 2903 | 33 | 1346 | 32 | - | - | 4693 | 1174 | -31 | 101 | -38 | 142 | ../test/100_Independent0663 |
| 100 | 1424 | 30 | 944 | 31 | - | - | 3516 | 1159 | -613 | 115 | -671 | 112 | ../test/100_Independent0664 |
| 100 | 2447 | 32 | 1021 | 45 | - | - | 3909 | 1199 | -238 | 97 | -277 | 163 | ../test/100_Independent0665 |

| Rozmiar instancji | Wartość kryterium sortowania po r_j | Czas działania algorytmu sortowania po r_j [µs] | Wartość kryterium sortowania po d_j | Czas działania algorytmu sortowania po d_j [µs] | Wartość kryterium przeglądu zupełnego | Czas działania przeglądu zupełnego [µs] | Wartość kryterium algorytmu własnego | Czas działania algorytmu własnego [µs] | Wartość kryterium algorytmu Schrage | Czas działania algorytmu Schrage [µs] | Wartość kryterium algorytmu Schrage wywłaszczającego | Czas działania algorytmu Schrage z wywłaszczaniem [µs] | Nazwa pliku z danymi testowymi |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 2379 | 32 | 1229 | 31 | - | - | 3780 | 1172 | -110 | 98 | -125 | 113 | ../test/100_Independent0666 |
| 100 | 2187 | 32 | 1090 | 32 | - | - | 4348 | 1202 | -337 | 166 | -358 | 197 | ../test/100_Independent0667 |
| 100 | 2173 | 31 | 821 | 33 | - | - | 4295 | 1193 | -207 | 97 | -209 | 110 | ../test/100_Independent0668 |
| 100 | 2246 | 32 | 1249 | 32 | - | - | 3723 | 1162 | -127 | 97 | -186 | 124 | ../test/100_Independent0669 |
| 100 | 2070 | 59 | 214 | 31 | - | - | 3320 | 1145 | -202 | 101 | -256 | 115 | ../test/100_Independent0670 |
| 100 | 1261 | 30 | -168 | 32 | - | - | 3056 | 1164 | -812 | 95 | -841 | 109 | ../test/100_Independent0671 |
| 100 | 2114 | 32 | 181 | 32 | - | - | 4280 | 1245 | -190 | 97 | -255 | 140 | ../test/100_Independent0672 |
| 100 | 1863 | 32 | -419 | 32 | - | - | 3391 | 1194 | -701 | 122 | -710 | 112 | ../test/100_Independent0673 |
| 100 | 1372 | 42 | -46 | 30 | - | - | 3192 | 1473 | -647 | 97 | -669 | 180 | ../test/100_Independent0674 |
| 100 | 2130 | 33 | 104 | 31 | - | - | 4111 | 1167 | -269 | 96 | -285 | 152 | ../test/100_Independent0675 |
| 100 | 2059 | 45 | 48 | 31 | - | - | 4458 | 1408 | -216 | 107 | -305 | 108 | ../test/100_Independent0676 |
| 100 | 1517 | 33 | -510 | 31 | - | - | 3360 | 1498 | -749 | 99 | -753 | 111 | ../test/100_Independent0677 |
| 100 | 1818 | 66 | 4 | 35 | - | - | 4150 | 1165 | -248 | 100 | -290 | 109 | ../test/100_Independent0678 |

Dla wskazanych algorytmów wyliczono błędy względem najlepszych znanych wartości z przykładów załączonych przez naukowców z uniwersytetu. Obliczenia wykonywano na instancjach o rozmiarze 100 zadań. Najmniejsze, największe i średnie błędy względne, a także pierwsze 5 obliczeń dla instancji zamieszczono w tabeli 1.

Tabela 1: Wyliczone błędy dla zaimplementowanych algorytmów

Błędy algorytmów

| Nazwa instancji | Najlepsza znana wartość | Błąd względny, rj | Błąd względny, dj | Błąd względny, own |
|---|---|---|---|---|
| ../test/100_Independent0600 | 1036 | 250,58% | 124,32% | 409,75% |
| ../test/100_Independent0601 | 728 | 294,92% | 141,07% | 450,41% |
| ../test/100_Independent0602 | 700 | 388,71% | 217,00% | 710,43% |
| ../test/100_Independent0603 | 1139 | 207,46% | 106,50% | 329,68% |
| ../test/100_Independent0604 | 952 | 233,93% | 122,06% | 418,70% |

| Algorytm | Błąd minimalny | Błąd maksymalny | Błąd średni |
|---|---|---|---|
| Algorytm sortowania po rj | 190,63% | 9132,00% | 1372,89% |
| Algorytm sortowania po dj | 0,00% | 9360,00% | 1000,88% |
| Algorytm własny | 276,92% | 16844,00% | 2434,83% |

# 5 Wnioski

- trudnością implementacyjną były przypadki graniczne, które wymagały wprowadzenia odpowiednich flag w kodzie
- algorytm własny, mimo że nie jest optymalny, znajduje rozwiązanie w rozsądnym czasie
- algorytm przeglądu zupełnego już dla kilku elementów wykonywał się stosunkowo długo
- czas wykonania przeglądu zupełnego rośnie wykładniczo co uniemożliwiło wykonanie testów już dla kilkunastu elementów w instancji
- algorytm sortowania po $d_j$ daje zauważalnie lepsze wyniki niż sortowanie po $r_j$
- implementacja algorytmu opartego na odpowiedniej heurystyce może znacząco poprawić otrzymywane wyniki, zarówno pod względem czasu obliczeń, jak i minimalizacji kryterium
- mimo posiadania sprzętu, który bardzo szybko wykonuje obliczenia, nie zawsze można rozwiązać koncepcyjnie proste zadanie w rozsądnym czasie
- przeprowadzanie ręcznej symulacji bardzo ułatwia implementację algorytmu
- dodanie mechanizmu wywłaszczania do algorytmu Schrage nieznacznie polepsza wyniki